

qprov: recording where a number came from

Patrick Taylor

What the engine records

When a paper states “the first nonzero coefficient appears at q^{46} ”, that sentence is the end of a long chain: some code ran, with some inputs, on some machine, at some version, and produced a number. Months later the chain is usually gone. **qprov** captures the chain automatically. Put one line, `@qprov.tracked`, above a function, and every call writes a record of the inputs, the output, the source code, the git commit, the machine, and the runtime. Later, `qprov verify <id>` re-runs the recorded call and confirms the output is identical, bit for bit. If it is not, you find out loudly.

How it works

Three ideas carry the design.

1. **Content-addressed computations.** Each recorded call gets an id equal to `blake2b(function name, input hash, code hash)`. The same inputs and the same code always produce the same id, so a quiet change to either is visible immediately. Files read by path are hashed by content, so two machines reading different data under the same filename can never collide silently.
2. **Exact serialization.** The store keeps a canonical JSON form with type tags, so large integers, exact rationals, and Laurent series round trip without loss. This matters because the numbers being tracked are exact algebraic objects, not floating-point approximations.
3. **Claims gated to evidence.** A claim is a one-line statement that ends up in the paper as a `\fact{...}` macro. A claim tagged with a paper must point at a recorded computation, so a paper-bound statement can never lose its backing.

The store is a local SQLite database and a folder of compressed payloads inside the project directory. There is no server and no account.

Worked example

The bundled demo (`example/run_example.py`) wraps a function that computes the q -deformed real $[x]_q$ to N coefficients, runs it for a few constants such as $[\pi]_q$ and $[\sqrt{2}]_q$, and records one claim per result, for instance that the first nonzero coefficient of $[\pi]_q$ after q^2 appears at q^{10} .

Each call is wrapped with `@qprov.tracked`, so the inputs, the result, the exact source code, and the machine are all recorded. Each claim is linked back to the computation that produced it, and `qprov verify` re-runs that computation on demand and checks the result against the store, bit for bit. A reader does not have to trust the number in the paper; they can re-run the receipt.

See `README.md` for the full command-line tool and the `@tracked` API.